

PHASE 01 / 07

Foundations & Setup

What databases, SQL and MySQL really are, how a query travels through the server, and how to build a working lab on your own machine.

Introduction | What is SQL & MySQL? | MySQL Environment Setup

You will finish this phase with a running MySQL server, the course's sales database loaded, and a clear mental model of where SQL sits in a data engineering pipeline.

CONTENTS

Inside this phase

Introduction: Why SQL? 3

- Data, database, DBMS, SQL: the ladder 3
- SQL is declarative: say what, not how 4
- Where SQL lives in a data pipeline 4
- Two kinds of database workloads: OLTP vs OLAP 5
- Your 7-phase roadmap 5

What is SQL and MySQL? 7

- The relational model in one picture 7
- SQL is a family of five sub-languages 8
- What is MySQL? 8
- What happens when you press Run? 9
- Writing SQL that other people can read 9
- MySQL vs other popular databases 10

MySQL Environment Setup 11

- Setup in five steps 11
- Option A: command line 12
- Option B: Docker (recommended for data engineers) 12
- Option C: MySQL Workbench 12
- What you are loading: the `sales` star schema 13
- Verify the installation 14
- Your first queries 15
- The small `ecom` practice tables 16
- When things go wrong: common errors 16

Phase 1 Recap & Practice 18

- Cheat sheet 18
- Check yourself 18
- Practice lab 19
- Answer key 19
- Up next 19

1.1 PART 1.1 · INTRODUCTION

Introduction: Why SQL?

Every data engineer, analyst and backend developer ends up in the same place: a database. SQL is the one skill that works in almost every data job, and it has barely changed in 40 years.

What you will learn

- Define data, database, DBMS and SQL, and how they stack
- Explain why SQL is called the bridge between humans and data
- See where SQL sits in a modern data pipeline
- Tell OLTP and OLAP systems apart
- Know the 7-phase roadmap you are about to follow

Why it matters for data engineering. Pipelines move data, but SQL is how data gets cleaned, joined, aggregated and checked. Most transformation work in warehouses (Snowflake, BigQuery, Redshift) and tools like dbt is written in SQL.

Data, database, DBMS, SQL: the ladder

These four words are used loosely, so let's pin them down. Each one builds on the one before it.

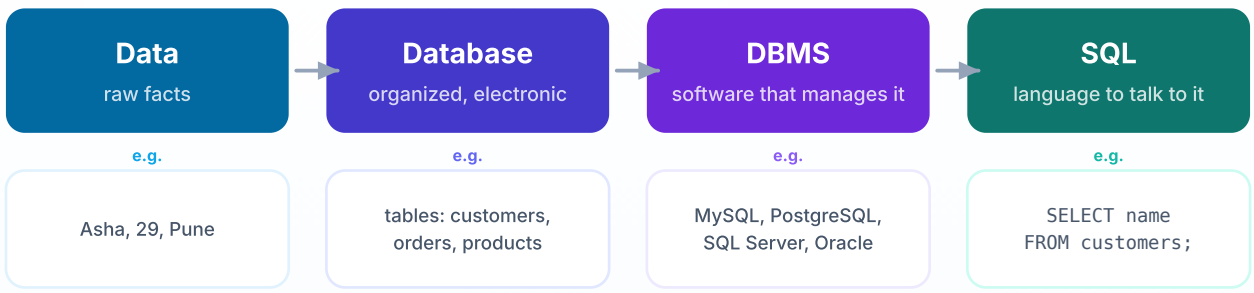


Figure 1. From raw facts to a language that asks questions about them. Each layer depends on the one on its left.

- **Data**: individual facts such as a name, a price, a timestamp. On its own it is just noise.
- **Database**: an organized collection of data stored electronically so it can be found, changed and protected.
- **DBMS** (Database Management System): the software that stores the database and answers requests. MySQL is a DBMS.
- **SQL** (Structured Query Language): the language you use to *store, retrieve and manipulate* data inside a DBMS.

REMEMBER

Your course puts it neatly: **SQL is the bridge between humans and data.** You write a request in near-English, and the database figures out how to get it done.

SQL is declarative: say what, not how

In a programming language you write the steps (loop, compare, collect). In SQL you describe the result you want and the database engine chooses the steps. That is why one short query can replace dozens of lines of code.

PYTHON — Imperative

```python
# Python: you control the HOW
result = []
for p in products:
    if p["category"] == "Clothing" and
p["unit_price"] > 500:
        result.append(p["product_name"])
result.sort()
```

SQL — Declarative

```sql
-- SQL: you describe the WHAT
SELECT product_name
FROM   dim_product
WHERE  category = 'Clothing'
  AND  unit_price > 500
ORDER  BY product_name;
```

Where SQL lives in a data pipeline

A data engineer's job is to move data from where it is created to where it is useful, reliably. SQL shows up at almost every stage after the data lands.



Figure 2. A simplified pipeline. The three dark boxes are where most day-to-day SQL is written: staging, transformation and the warehouse.

Two kinds of database workloads: OLTP vs OLAP

MySQL was built for **OLTP** (Online Transaction Processing): the database behind an app, handling thousands of small reads and writes. Data engineers also build **OLAP** (Online Analytical Processing) systems, such as warehouses, that scan huge amounts of data to answer questions. The SQL you learn here works in both worlds.

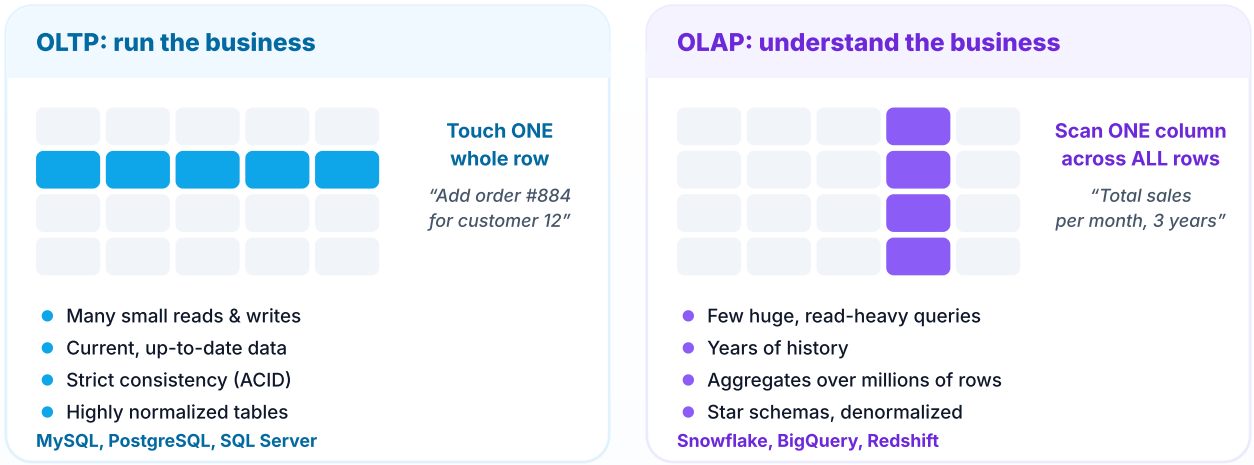


Figure 3. OLTP touches whole rows; OLAP scans whole columns. Same SQL language, very different engineering.

DATA-ENGINEERING LENS

Your practice database (`sales`) is shaped like an **OLAP star schema**: one big fact table (`fact_sales`) surrounded by dimension tables. You will meet this design in Phase 2 and use it in every phase after.

Your 7-phase roadmap

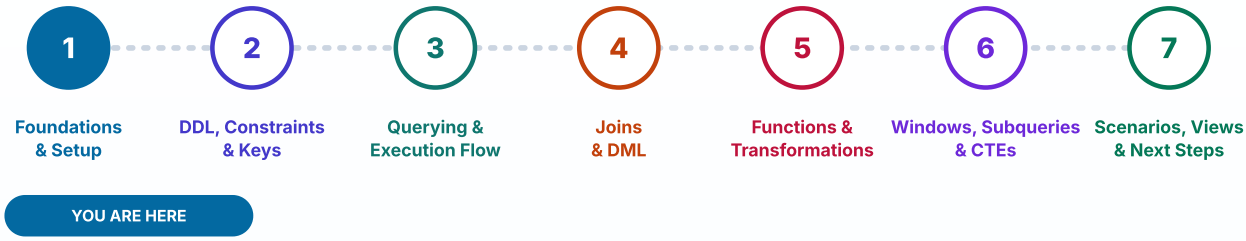


Figure 4. Seven PDFs, each building on the last. By Phase 7 you can design tables, write analytical queries and package logic into reusable database objects.

Phase	Course topics covered	Core skill you gain
1	Introduction · What is SQL/MySQL · Environment setup	Mental model + working lab
2	DDL commands · Constraints · Keys	Design tables that protect data quality
3	SQL basic commands · SQL execution flow	Filter, sort, group, and know *why* a query behaves as it does

Phase	Course topics covered	Core skill you gain
4	SQL joins · DML commands	Combine tables; insert, update and delete safely
5	Numeric, string, date functions · Casting · CASE	Clean and transform columns
6	Window & ranking functions · Subqueries · CTEs	Analytical SQL: rankings, running totals, readable pipelines
7	Real-time scenarios · Views · Procedures · Functions · Next steps	Production patterns and your DE roadmap

1.2 PART 1.2 · CONCEPTS

What is SQL and MySQL?

Before typing commands, it helps to know the vocabulary and what happens inside the server when you press Run. This chapter gives you both.

What you will learn

- Describe the relational model: tables, rows, columns, keys
- Group SQL commands into DDL, DML, DQL, DCL and TCL
- Trace a query through MySQL's server architecture
- Compare MySQL with other popular databases
- Follow clean SQL style and naming conventions

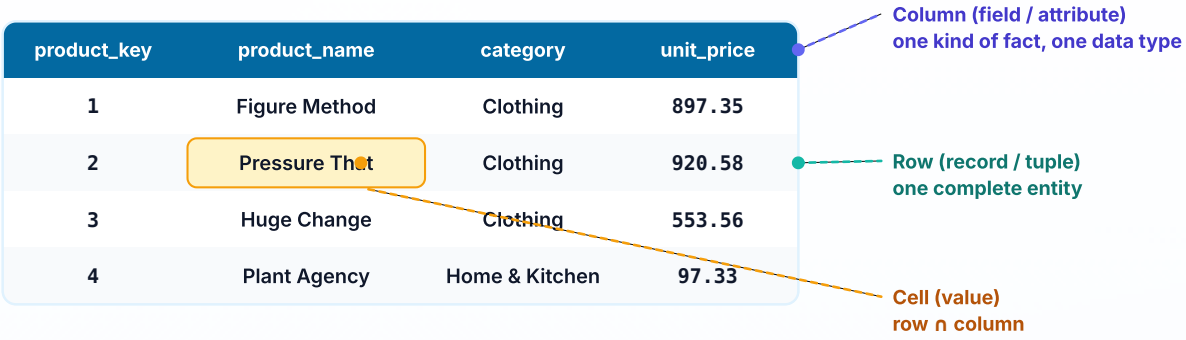
Why it matters for data engineering. Nearly every warehouse speaks a dialect of SQL. If you know the relational model and the command families, moving from MySQL to Snowflake or BigQuery is a matter of small syntax differences, not a new skill.

The relational model in one picture

MySQL is a **relational** database management system (RDBMS). Data lives in **tables**; each table is a grid of **rows** and **columns**. Different tables are linked through shared values called **keys**. This idea, proposed by E. F. Codd in 1970, is still the backbone of most business data.

Table name: dim_product

product_key	product_name	category	unit_price
1	Figure Method	Clothing	897.35
2	Pressure That	Clothing	920.58
3	Huge Change	Clothing	553.56
4	Plant Agency	Home & Kitchen	97.33



product_key is the primary key: a unique ID that other tables can point to.

Real rows from your course database.

Figure 5. Anatomy of a table, using the first rows of `dim_product`. The same words (row, column, key) will appear in every later phase.

Everyday term	Database terms	Meaning
Spreadsheet sheet	Table / relation	A named collection of rows that share the same columns
A row	Row / record / tuple	One thing: one product, one customer, one sale
A column header	Column / field / attribute	A property, with a fixed data type (INT, VARCHAR, DATE ...)
Folder of sheets	Database / schema	A container of related tables. In MySQL, *database* and *schema* are synonyms
VLOOKUP key	Primary & foreign key	Values that identify a row and connect it to rows in other tables

SQL is a family of five sub-languages

Your course introduces them one chapter at a time. It helps to see the whole family first, because every command you meet belongs to exactly one of these.

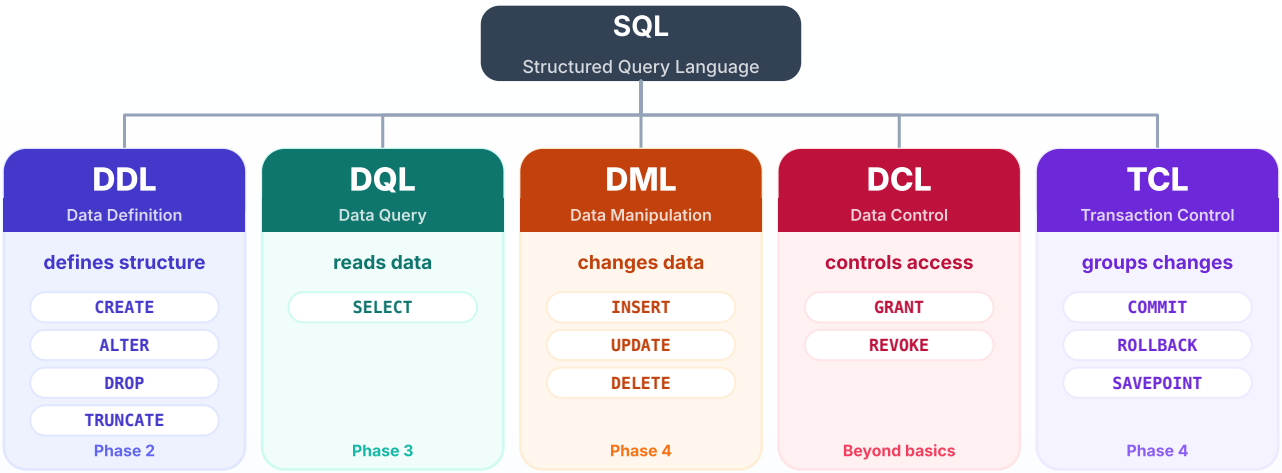


Figure 6. Five command families. Some books fold SELECT into DML; treating it as its own group (DQL) matches how you will learn it.

> **NOTE**
> Your course slides split the material the same way: **DDL commands** (CREATE, ALTER, DROP, TRUNCATE), then querying, then **DML commands** (INSERT, UPDATE, DELETE). Notes for each arrive in the phase shown under each box.

What is MySQL?

- **Open-source RDBMS** created in 1995 by MySQL AB, now owned by Oracle. One of the most widely deployed databases on the web.
- **Client-server**: the server process (`mysqld`) holds the data; clients such as Workbench, Python scripts or Airflow connect to it, by default on **port 3306**.

- **Pluggable storage engines**. **InnoDB** is the default: it supports transactions, foreign keys and row-level locking.
- **MySQL 8 added** window functions, CTEs, `CHECK` constraints and more. You need these for Phases 5 to 7, so use MySQL **8.0 or newer**.

MYSQL SPECIFIC

MariaDB is a community fork of MySQL and is largely compatible for everything in this course. Every example in these notes was executed on a live server and the output shown is real.

What happens when you press Run?

Whenever you send a query, it travels through the same five stages. Knowing them explains later topics like indexes, the execution flow of a SELECT, and why some queries are slow.

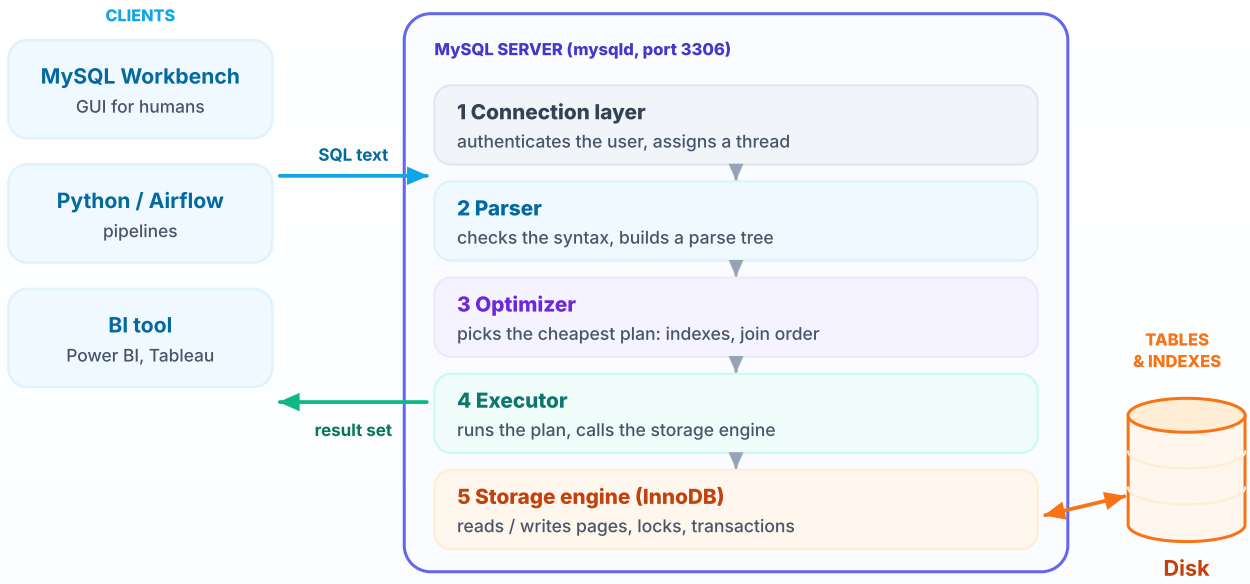


Figure 7. Request path through MySQL. The optimizer (step 3) is the part that decides *how* to run your declarative query.

Writing SQL that other people can read

Style is not decoration. In a data team, your query will be read, reviewed and re-run by someone else, so the habits below pay off immediately.

Keywords UPPERCASE

`SELECT`, `FROM`, `WHERE`. The engine does not care; humans scan faster.

snake_case names

`unit_price`, `join_date`. Avoid spaces and keywords as identifiers.

One clause per line

Put `FROM`, `WHERE`, `GROUP BY` on their own lines and indent conditions.

SQL — Readable layout

```sql
-- Single-line comment (note the space after --)
# MySQL also allows hash comments
/* Multi-line
   comment */

SELECT  customer_id,          -- one column per line
        email
FROM    dim_customer
WHERE   country = 'France'    -- strings use single quotes
LIMIT   5;                    -- every statement ends with ;
```

WATCH OUT

On Linux, MySQL treats **table names as case-sensitive** (`Orders` ≠ `orders`); on Windows it does not. Always use consistent lowercase names so your SQL runs on any machine.

MySQL vs other popular databases

Database	Type	Where you will meet it	Notes vs MySQL
MySQL	OLTP RDBMS	Web apps, startups, WordPress, SaaS back-ends	Simple, fast reads, huge community
PostgreSQL	OLTP RDBMS	Modern apps, analytics, data platforms	More advanced SQL features, richer types; `FULL JOIN` is supported
SQL Server	OLTP RDBMS	Enterprises on Microsoft stack	T-SQL dialect: `TOP` , `ISNULL` , brackets
Oracle	OLTP RDBMS	Banks, large enterprises	PL/SQL, `ROWNUM` , `DUAL`
Snowflake / BigQuery / Redshift	Cloud OLAP warehouse	Analytics & data engineering teams	Column storage; ANSI SQL plus extras (`QUALIFY` , arrays)
SQLite	Embedded	Mobile, tests, local scripts	A single file, no server

INTERVIEW ANGLE

“Is SQL the same as MySQL?” **No.** SQL is the *language* (a standard). MySQL is one *product* that implements it, with its own extras. Saying this crisply is a classic interview opener.

1.3 PART 1.3 · HANDS-ON

MySQL Environment Setup

Now build the lab you will use for all seven phases. You need three things: a MySQL server, a client to type queries into, and some data to query.

What you will learn

- Install MySQL Server and a client (Workbench or CLI)
- Run MySQL in Docker as an alternative setup
- Load the course database from `source.sql`
- Understand the `sales` star schema you will query throughout
- Run your first queries and fix common errors

Why it matters for data engineering. Engineers rarely work on a hosted GUI. Being comfortable with a local server, the `mysql` command-line client and a Docker container is the same workflow you will use to test pipelines on your laptop before they touch production.

Setup in five steps

1. **Install MySQL Server (8.0 or newer)**
   Windows: the *MySQL Installer* from mysql.com (choose "Server only" or "Developer Default"). macOS: `brew install mysql` or the DMG. Ubuntu/Debian: `sudo apt install mysql-server`. Set a **root password** and keep the default port **3306**.

2. **Install a client**
   MySQL Workbench is a free GUI with an editor, results grid and diagram tools. VS Code with a MySQL extension or DBeaver also work. The command-line client `mysql` is already included with the server.

3. **Connect**
   Open Workbench, click the **+** next to *MySQL Connections*, use host `127.0.0.1`, port `3306`, user `root`, then **Test Connection** and store the password.

4. **Load the course data**
   Create the `sales` database and run the course file `Source_Setup/source.sql` against it (commands on the next page).

5. **Verify**
   Run `SHOW TABLES;` and a couple of `COUNT(*)` checks. You should see five tables and exactly 5,000 rows in `fact_sales`.

Option A: command line

SHELL + SQL — Terminal

```
# 1) log in (you will be asked for the root password)
mysql -u root -p

-- 2) inside the mysql> prompt
CREATE DATABASE sales;
USE sales;
SOURCE /full/path/to/Source_Setup/source.sql;    -- loads 5 tables + 6,785 INSERTs

-- or, from the OS shell without entering the prompt:
--   mysql -u root -p sales < Source_Setup/source.sql
```

Option B: Docker (recommended for data engineers)

A container gives you a throw-away database: break it, delete it, recreate it in seconds. It also matches how databases are run in CI pipelines and cloud dev environments.

SHELL — Docker

```
docker run --name mysql-lab -e MYSQL_ROOT_PASSWORD=root123 \
           -p 3306:3306 -v "$PWD":/scripts -d mysql:8

# wait ~20 seconds for the server to start, then:
docker exec -it mysql-lab mysql -uroot -proot123 -e "CREATE DATABASE sales;"
docker exec -i  mysql-lab sh -c 'mysql -uroot -proot123 sales < /scripts/Source_Setup/source.sql'
docker exec -it mysql-lab mysql -uroot -proot123 sales        # interactive session
```

Option C: MySQL Workbench

- **File → Open SQL Script...** and pick `source.sql`.
- Run `CREATE DATABASE sales;` first, then **double-click** `sales` in the Schemas panel. It turns **bold**, which means it is the default schema.
- Execute with the lightning bolt (`Ctrl` + `Shift` + `Enter` runs the whole script; `Ctrl` + `Enter` runs the current statement).
- Click the refresh icon in the Schemas panel. The five tables appear.

PRO TIP

In Workbench, if you see **Error 1046: No database selected**, you forgot to set the default schema. Fix it with `USE sales;` at the top of your script.

What you are loading: the `sales` star schema

The course database models a retail business. One central **fact table** records every sale; four **dimension tables** describe the who, what, where and when. This layout, called a **star schema**, is the standard way warehouses are designed.

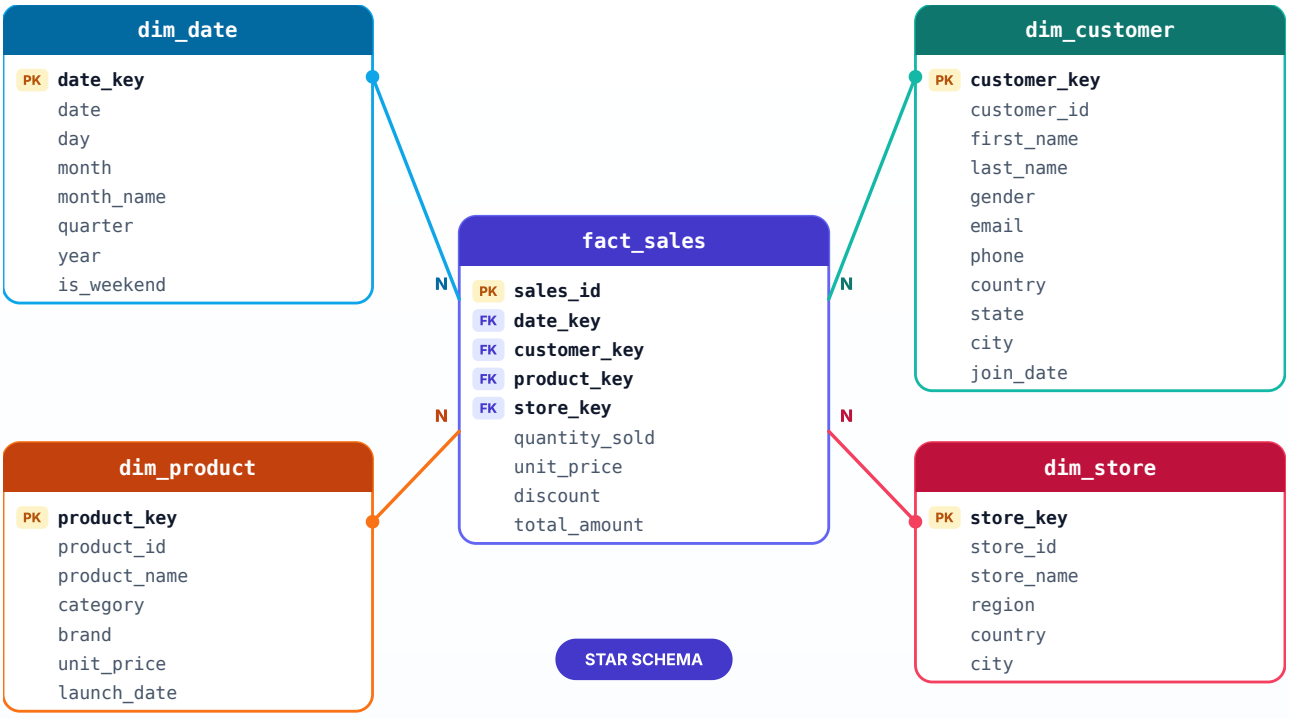


Figure 8. Entity-relationship view of the course database. PK = primary key, FK = foreign key; the dot marks the "one" side, N the "many" side.

Table	Role	Rows	Grain (one row = ...)
dim_date	Dimension: when	730	one calendar day
dim_customer	Dimension: who	500	one customer
dim_product	Dimension: what	500	one product
dim_store	Dimension: where	50	one store
fact_sales	Fact: the event	5,000	one sale line (a product sold in a store to a customer on a date)

Verify the installation

SQL — Which tables exist?

```sql
SHOW TABLES;
```

RESULT — 5 ROWS

Tables_in_sales
dim_customer
dim_date
dim_product
dim_store
fact_sales

SQL — Row counts: a classic first sanity check after any data load

```sql
SELECT 'dim_date' AS table_name, COUNT(*) AS total_rows FROM dim_date
UNION ALL SELECT 'dim_customer', COUNT(*) FROM dim_customer
UNION ALL SELECT 'dim_product',  COUNT(*) FROM dim_product
UNION ALL SELECT 'dim_store',    COUNT(*) FROM dim_store
UNION ALL SELECT 'fact_sales',   COUNT(*) FROM fact_sales;
```

RESULT — 5 ROWS

table_name	total_rows
dim_date	730
dim_customer	500
dim_product	500
dim_store	50
fact_sales	5000

SQL — Column names and types of one table

```sql
DESCRIBE dim_product;
```

RESULT — 7 ROWS

Field	Type	Null	Key	Default	Extra
product_key	int	NO	PRI	*NULL*	
product_id	varchar(20)	YES		*NULL*	
product_name	varchar(100)	YES		*NULL*	
category	varchar(50)	YES		*NULL*	
brand	varchar(50)	YES		*NULL*	
unit_price	decimal(10,2)	YES		*NULL*	

... 1 more rows not shown

DATA-ENGINEERING LENS

Row-count reconciliation is the first test every data engineer runs after loading data: does the target hold the same number of rows the source said it sent? Make `COUNT(*)` your reflex.

Your first queries

SQL — SELECT needs no table to compute an expression

```sql
SELECT 'Hello, SQL!' AS greeting, 2 + 3 AS five;
```

RESULT — 1 ROW

greeting	five
Hello, SQL!	5

SQL — Peek at a table: * means all columns, LIMIT keeps it small

```sql
SELECT * FROM dim_product LIMIT 5;
```

RESULT — 5 ROWS

product_key	product_id	product_name	category	brand	unit_price	launch_date
1	PROD0001	Figure Method	Clothing	BrandC	897.35	2020-12-06
2	PROD0002	Pressure That	Clothing	BrandD	920.58	2023-05-07
3	PROD0003	Huge Change	Clothing	BrandA	553.56	2025-07-03
4	PROD0004	Plant Agency	Home & Kitchen	BrandA	97.33	2023-12-24
5	PROD0005	Fast Real	Home & Kitchen	BrandE	333.97	2025-09-28

SQL — Pick columns, filter rows, sort

```sql
SELECT product_name, unit_price FROM dim_product WHERE unit_price > 990 ORDER BY unit_price DESC;
```

RESULT — 1 ROW

product_name	unit_price
Likely Brother	999.04

PRO TIP

Use `LIMIT` whenever you explore a table you have not seen before. On a billion-row table, `SELECT *` without it can take minutes and cost real money in cloud warehouses.

The small `ecom` practice tables

The course also uses tiny tables, `customers` and `orders`, to demonstrate joins and DML, where you can see every row at once. Create them in a second database:

SQL — Tiny tables for Phases 4 to 7

```sql
CREATE DATABASE IF NOT EXISTS ecom;
USE ecom;

CREATE TABLE customers (id INT, name VARCHAR(50), email VARCHAR(50));
INSERT INTO customers VALUES (101,'love','aa'), (201,'ansh','bb'), (301,'lamba','cc');

CREATE TABLE orders (o_id INT, cust_id INT, price INT);
INSERT INTO orders VALUES (1,101,1000), (2,201,1100), (3,501,1200);
```

SQL — ecom.customers

```sql
SELECT * FROM customers;
```

RESULT — 3 ROWS

id	name	email
101	love	aa
201	ansh	bb
301	lamba	cc

SQL — ecom.orders: order 3 belongs to customer 501, who does not exist in customers (on purpose!)

```sql
SELECT * FROM orders;
```

RESULT — 3 ROWS

o_id	cust_id	price
1	101	1000
2	201	1100
3	501	1200

When things go wrong: common errors

Error	Meaning	Fix
ERROR 2003: Can't connect to MySQL server on '127.0.0.1:3306'	Server is not running or the port is wrong	Start the MySQL service (or `docker start mysql-lab`); check port 3306
ERROR 1045: Access denied for user 'root'	Wrong password	Re-enter the password you set during installation

Error	Meaning	Fix
ERROR 1046: No database selected	No default schema	Run `USE sales;` or double-click the schema in Workbench
ERROR 1049: Unknown database 'sale'	Typo in the database name	`SHOW DATABASES;` to see exact names
ERROR 1146: Table 'sales.dim_prod' doesn't exist	Typo, wrong database, or script not loaded	`SHOW TABLES;` and re-run `source.sql`
ERROR 1064: You have an error in your SQL syntax	The parser could not understand the statement	Check commas, quotes, parentheses near the text MySQL quotes

1.4 WRAP-UP

Phase 1 Recap & Practice

Lock in the vocabulary, then prove the setup works by completing the lab below.

Cheat sheet

Concept	One-line definition
Database / DBMS	Organized stored data / the software (MySQL) that manages it
SQL	Declarative language to define, query and change data in a relational DBMS
RDBMS	Stores data in related tables of rows and columns
DDL · DQL · DML · DCL · TCL	Define structure · Query · Change data · Control access · Control transactions
Client / server	Workbench, Python, BI tools send SQL to `mysqld` on port 3306
Query path	Connect → Parse → Optimize → Execute → Storage engine → Result
OLTP vs OLAP	Many small transactional writes vs few large analytical scans
Star schema	One fact table surrounded by dimension tables (`sales` database)
Useful commands	`SHOW DATABASES;` `USE sales;` `SHOW TABLES;` `DESCRIBE t;` `SELECT … LIMIT n;`

Check yourself

Q1 Why is SQL called declarative?

You state the result you want; the optimizer decides the steps (indexes, join order) to produce it.

Q2 What is the difference between SQL and MySQL?

SQL is the standard language; MySQL is a database product that implements it with its own extensions.

Q3 Which sub-language does `TRUNCATE` belong to, and which does `SELECT`?

`TRUNCATE` is DDL (it changes table structure/storage). `SELECT` is DQL.

Q4 In the `sales` star schema, which table is the fact table and why?

`fact_sales`: it stores the measurable events (quantity, discount, total) and holds foreign keys to every dimension.

Q5 Name two places a data engineer uses SQL.

Transforming raw staging data into modeled warehouse tables; validating loads (row counts, duplicates, nulls); building views and stored procedures for BI consumers.

Practice lab

Run each task in your own MySQL. Answers are on the next page; try first.

1. Show every database on your server, then switch to `sales`.
2. List the tables and describe `dim_customer`. How many columns does it have?
3. How many rows are in `dim_customer`? In `fact_sales`?
4. Show the first 3 rows of `dim_store`, with only `store_name` and `country`.
5. Return `'Hello'` and today's date in one query (hint: `CURDATE()`).
6. Find the 5 most expensive products (`dim_product`) and show their name and price.

Answer key

SQL — Solutions

```sql
-- 1
SHOW DATABASES;
USE sales;
-- 2
SHOW TABLES;
DESCRIBE dim_customer;                    -- 11 columns
-- 3
SELECT COUNT(*) FROM dim_customer;        -- 500
SELECT COUNT(*) FROM fact_sales;          -- 5000
-- 4
SELECT store_name, country FROM dim_store LIMIT 3;
-- 5
SELECT 'Hello' AS msg, CURDATE() AS today;
-- 6
SELECT product_name, unit_price
FROM   dim_product
ORDER  BY unit_price DESC
LIMIT  5;
```

Up next

COMING IN PHASE 2

Phase 2: Table Design. You will create, alter and drop tables (DDL), protect data with constraints, and learn the six kinds of keys that tie a database together.